

Savitribai Phule Pune University
T.Y.B.Sc.(C.S.) (NEP – 2020) SEM – V Practical Examination
Lab Course CS-305-MJ-P
Operating System-I Duration:

Q 1) Write a C Menu driven Program to implement following functionality
a) Accept Available b) Display Allocation, Max c) Display the contents of need matrix d) Display Available

Process	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P0	2	3	2	9	7	5	3	3	2
P1	4	0	0	5	2	2			
P2	5	0	4	1	0	4			
P3	4	3	3	4	4	4			
P4	2	2	4	6	5	5			

Answer:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int ch, i, j;
```

```
    int alloc[5][3] = {  
        {2,3,2}, {4,0,0}, {5,0,4},  
        {4,3,3}, {2,2,4}  
    };
```

```
    int max[5][3] = {
```

```

    {9,7,5}, {5,2,2}, {1,0,4},
    {4,4,4}, {6,5,5}
};

int avail[3], need[5][3];

do
{
    printf("\n1.Accept Available");
    printf("\n2.Display Allocation & Max");
    printf("\n3.Display Need");
    printf("\n4.Display Available");
    printf("\n5.Exit");
    printf("\nEnter choice: ");
    scanf("%d",&ch);

    switch(ch)
    {
        case 1:
            printf("Enter Available A B C: ");
            scanf("%d%d%d",&avail[0],&avail[1],&avail[2]);

            for(i=0;i<5;i++)
                for(j=0;j<3;j++)
                    need[i][j]=max[i][j]-alloc[i][j];
            break;

        case 2:
            printf("\nP\tAllocation\tMax\n");
            for(i=0;i<5;i++)
                printf("P%d\t%d %d %d\t\t%d %d %d\n",
                    i,alloc[i][0],alloc[i][1],alloc[i][2],
                    max[i][0],max[i][1],max[i][2]);
            break;

        case 3:

```

```

    printf("\nNeed Matrix\n");
    for(i=0;i<5;i++)
        printf("P%d\t%d %d %d\n",i,
            need[i][0],need[i][1],need[i][2]);
    break;

case 4:
    printf("\nAvailable: %d %d %d\n",
        avail[0],avail[1],avail[2]);
    break;

case 5:
    printf("Exit");
    break;

default:
    printf("Invalid choice");
}
} while(ch != 5);

return 0;
}

```

Q.2) Write a C program that behaves like a shell which displays the command

prompt '\$'. It accepts the command, tokenize the command line and execute it by creating the child process. Also implement the additional command 'count' as

- a. \$ count c filename: It will display the number of characters in given file
- b. \$ count w filename: It will display the number of words in given file
- c. \$ count l filename: It will display the number of lines in given

file

Answer:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    char cmd[100], *a[10];
    int i;

    while (1)
    {
        printf("$ ");
        fflush(stdout);

        if (fgets(cmd, 100, stdin) == NULL)
            break;

        cmd[strcspn(cmd, "\n")] = '\0';

        if (strlen(cmd) == 0)
            continue;

        if (strcmp(cmd, "exit") == 0)
            break;

        i = 0;
        a[i] = strtok(cmd, " ");

        while (a[i] != NULL && i < 9)
            a[++i] = strtok(NULL, " ");
```

```

if (strcmp(a[0], "count") == 0)
{
    FILE *fp;
    char ch;
    int count = 0;

    if (a[1] == NULL || a[2] == NULL)
    {
        printf("Invalid command\n");
        continue;
    }

    fp = fopen(a[2], "r");

    if (fp == NULL)
    {
        printf("File not found\n");
        continue;
    }

    while ((ch = fgetc(fp)) != EOF)
    {
        if (a[1][0] == 'c')
            count++;

        else if (a[1][0] == 'l' && ch == '\n')
            count++;

        else if (a[1][0] == 'w')
        {
            static int inword = 0;

            if (ch == ' ' || ch == '\n' || ch == '\t')
                inword = 0;
            else if (!inword)

```

```

        {
            count++;
            inword = 1;
        }
    }
}

if (a[1][0] == 'c')
    printf("Characters = %d\n", count);
else if (a[1][0] == 'w')
    printf("Words = %d\n", count);
else if (a[1][0] == 'l')
    printf("Lines = %d\n", count);

fclose(fp);
}
else
{
    if (fork() == 0)
    {
        execvp(a[0], a);
        exit(1);
    }
    wait(NULL);
}
}

return 0;
}

```

Q.3. Oral/Viva

Savitribai Phule Pune University
T.Y.B.Sc.(C.S.) (NEP – 2020) SEM – V Practical Examination
Lab Course CS-305-MJ-P
Operating System-I Duration:

Q.1) Write a program to simulate Linked file allocation method. Assume disk with 'n' number of blocks.

Give value of 'n' as input. Randomly mark some block as allocated and accordingly maintain the list of free blocks Write menu driver program with menu options as mentioned below and implement each option.

- ☐ Show Bit Vector
- ☐ Create New File
- ☐ Show Directory
- ☐ Exit

Answer:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
```

```
int main()
{
    int n, i, ch, f, size;
    int block[100] = {0};
    char name[20];

    printf("Enter number of blocks: ");
    scanf("%d", &n);

    srand(time(0));
```

```

/* Randomly allocate some blocks */
for(i = 0; i < n; i++)
    block[i] = rand() % 2;

while(1)
{
    printf("\n1. Show Bit Vector");
    printf("\n2. Create New File");
    printf("\n3. Show Directory");
    printf("\n4. Exit");
    printf("\nEnter choice: ");
    scanf("%d", &ch);

    switch(ch)
    {
        case 1:
            printf("\nBit Vector:\n");
            for(i = 0; i < n; i++)
                printf("%d ", block[i]);
            printf("\n");
            break;

        case 2:
            printf("Enter file name: ");
            scanf("%s", name);

            printf("Enter number of blocks: ");
            scanf("%d", &size);

            printf("Enter starting block: ");
            scanf("%d", &f);

            printf("File %s allocated at: ", name);

            for(i = f; i < f + size; i++)

```

```

    {
        if(i >= n || block[i] == 1)
        {
            printf("\nBlock not free!\n");
            break;
        }

        block[i] = 1;
        printf("%d ", i);
    }
    printf("\n");
    break;

case 3:
    printf("\nDirectory:\n");
    printf("File: %s\n", name);
    printf("Starting Block: %d\n", f);
    printf("Number of Blocks: %d\n", size);
    break;

case 4:
    exit(0);

default:
    printf("Invalid choice\n");
}
}

return 0;
}

```

Q.2) Write a simulation program for disk scheduling using FCFS algorithm. Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of requests in the

order in which it is served. Also display the total number of head moments.
55, 58, 39, 18, 90, 160, 150, 38, 184
Start Head Position: 50

Answer:

```
#include <stdio.h>

int main()
{
    int n, i, ch, start, size;
    int block[100] = {0};
    char file[20];

    scanf("%d", &n);

    do
    {
        printf("\n1. Show Bit Vector");
        printf("\n2. Create New File");
        printf("\n3. Show Directory");
        printf("\n4. Exit");
        printf("\nEnter choice: ");
        scanf("%d", &ch);

        switch(ch)
        {
            case 1:
                printf("Bit Vector: ");
                for(i=0;i<n;i++)
                    printf("%d ",block[i]);
                printf("\n");
                break;

            case 2:
                printf("Enter file name: ");
```

```
scanf(" %s",file);
```

```
printf("Enter starting block: ");  
scanf(" %d",&start);
```

```
printf("Enter number of blocks: ");  
scanf(" %d",&size);
```

```
for(i=start;i<start+size;i++)  
    block[i]=1;
```

```
printf("File created successfully\n");  
break;
```

```
case 3:
```

```
    printf("File: %s\n",file);  
    printf("Start: %d\n",start);  
    printf("Blocks: %d\n",size);  
    break;
```

```
case 4:
```

```
    printf("Exit");  
    break;
```

```
default:
```

```
    printf("Invalid choice\n");
```

```
}
```

```
}while(ch!=4);
```

```
return 0;
```

```
}
```

Q.3. Oral/Viva

Savitribai Phule Pune University
T.Y.B.Sc.(C.S.) (NEP – 2020) SEM – V Practical Examination
Lab Course CS-305-MJ-P
Operating System-I Duration:

Q 1) Write a C program to simulate Banker's algorithm for the purpose of deadlock avoidance. Consider the following snapshot of system, A, B, C and D is the resource type.

Process	Allocation				Max				Available			
	A	B	C	D	A	B	C	D	A	B	C	D
P0	0	0	1	2	0	0	1	2	1	5	2	0
P1	1	0	0	0	1	7	5	0				
P2	1	3	5	4	2	3	5	6				
P3	0	6	3	2	0	6	5	2				
P4	0	0	1	4	0	6	5	6				

- Calculate and display the content of need matrix?
- Is the system in safe state? If display the safe sequence

Answer:

```
#include <stdio.h>
```

```
int main()
{
    int alloc[5][4] = {
        {0,0,1,2},
        {1,0,0,0},
        {1,3,5,4},
        {0,6,3,2},
        {0,0,1,4}
    };

```

```
    int max[5][4] = {
        {0,0,1,2},

```

```

    {1,7,5,0},
    {2,3,5,6},
    {0,6,5,2},
    {0,6,5,6}
};

int avail[4] = {1,5,2,0};
int need[5][4], finish[5] = {0};
int safe[5], i, j, k, count = 0, found;

/* Need = Max - Allocation */
for(i=0;i<5;i++)
    for(j=0;j<4;j++)
        need[i][j] = max[i][j] - alloc[i][j];

printf("Need Matrix:\n");
for(i=0;i<5;i++)
{
    printf("P%d : ",i);
    for(j=0;j<4;j++)
        printf("%d ",need[i][j]);
    printf("\n");
}

/* Banker's Algorithm */
while(count < 5)
{
    found = 0;

    for(i=0;i<5;i++)
    {
        if(finish[i] == 0)
        {
            for(j=0;j<4;j++)
                if(need[i][j] > avail[j])
                    break;

```

```

        if(j == 4)
        {
            for(k=0;k<4;k++)
                avail[k] += alloc[i][k];

            safe[count++] = i;
            finish[i] = 1;
            found = 1;
        }
    }
}

if(found == 0)
    break;
}

if(count == 5)
{
    printf("\nSystem is in SAFE state\n");
    printf("Safe Sequence: ");

    for(i=0;i<5;i++)
        printf("P%d ",safe[i]);
}
else
    printf("\nSystem is NOT in SAFE state");

return 0;
}

```

Q 2) Write a simulation program for disk scheduling using SSTF algorithm.
 Accept total number of disk blocks, disk request string, and current head

position from the user. Display the list of requests in the order in which it is served. Also display the total number of head moments.

98, 183, 37, 122, 14, 124, 65

Start Head Position: 53

Answer:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int n, head, i, j, min, pos, total=0;
    int a[100], used[100]={0};

    scanf("%d",&n);

    for(i=0;i<n;i++)
        scanf("%d",&a[i]);

    scanf("%d",&head);

    printf("Order: %d",head);

    for(i=0;i<n;i++)
    {
        min=9999;

        for(j=0;j<n;j++)
        {
            if(!used[j] && abs(head-a[j])<min)
            {
                min=abs(head-a[j]);
                pos=j;
            }
        }
    }
```

```
    used[pos]=1;
    total+=abs(head-a[pos]);
    head=a[pos];

    printf(" -> %d",head);
}

printf("\nTotal head movement = %d",total);

return 0;
}
```

Q.3. Oral/Viva

Savitribai Phule Pune University
T.Y.B.Sc.(C.S.) (NEP – 2020) SEM – V Practical Examination
Lab Course CS-305-MJ-P
Operating System-I Duration:

Q 1) Write the simulation program for demand paging and show the page scheduling
total and number of page faults according the FIFO page replacement
algorithm. Assume the memory of n frames.
Reference String: 3,4,5,6,3,4,7,3,4,5,6,7,2,4,6

Answer: #include <stdio.h>

```
int main()
{
    int frames[20], n, pages[] = {3, 4, 5, 6, 3, 4, 7, 3, 4, 5, 6, 7, 2, 4, 6};
    int pageCount = sizeof(pages) / sizeof(pages[0]);
    int i, j, k, pointer = 0;
    int pageFaults = 0, hit;

    printf("Enter number of frames: ");
    scanf("%d", &n);

    // Initialize frames
    for (i = 0; i < n; i++)
        frames[i] = -1;

    printf("\nFIFO Page Replacement\n");
    printf("Reference String: ");

    for (i = 0; i < pageCount; i++)
        printf("%d ", pages[i]);

    printf("\n\n");
```

```

printf("Page\tFrames\t\tStatus\n");
printf("-----\n");

for (i = 0; i < pageCount; i++)
{
    hit = 0;

    // Check whether page is already in memory
    for (j = 0; j < n; j++)
    {
        if (frames[j] == pages[i])
        {
            hit = 1;
            break;
        }
    }

    // Page fault
    if (hit == 0)
    {
        frames[pointer] = pages[i];
        pointer = (pointer + 1) % n;
        pageFaults++;
    }

    // Display current frame contents
    printf("%d\t", pages[i]);

    for (k = 0; k < n; k++)
    {
        if (frames[k] == -1)
            printf("- ");
        else
            printf("%d ", frames[k]);
    }
}

```

```

    if (hit)
        printf("\tHit");
    else
        printf("\tPage Fault");

    printf("\n");
}

printf("\nTotal number of page faults = %d\n", pageFaults);
printf("Total number of page references = %d\n", pageCount);
printf("Number of page hits = %d\n", pageCount - pageFaults);

return 0;
}

```

Q 2) Consider a system with ‘n’ processes and ‘m’ resource types. Accept number of instances for every resource type. For each process accept the allocation and maximum requirement matrices. Write a program to display the contents of need matrix and to check if the given request of a process can be granted immediately or not. (Use resource request algorithm)

Answer:

```

#include <stdio.h>

int main()
{
    int n, m, i, j, p;
    printf("Enter processes and resources: ");
    scanf("%d%d", &n, &m);

    int a[n][m], max[n][m], need[n][m];
    int total[m], avail[m], req[m], work[m], finish[n];

```

```
printf("Enter total resources: ");  
for(j=0;j<m;j++) scanf("%d",&total[j]);
```

```
printf("Enter Allocation Matrix:\n");  
for(i=0;i<n;i++)  
    for(j=0;j<m;j++) scanf("%d",&a[i][j]);
```

```
printf("Enter Maximum Matrix:\n");  
for(i=0;i<n;i++)  
    for(j=0;j<m;j++) {  
        scanf("%d",&max[i][j]);  
        need[i][j]=max[i][j]-a[i][j];  
    }
```

```
/* Calculate Available */  
for(j=0;j<m;j++) {  
    avail[j]=total[j];  
    for(i=0;i<n;i++) avail[j]-=a[i][j];  
}
```

```
printf("\nNeed Matrix:\n");  
for(i=0;i<n;i++) {  
    for(j=0;j<m;j++) printf("%d ",need[i][j]);  
    printf("\n");  
}
```

```
printf("\nEnter process number: ");  
scanf("%d",&p);
```

```
printf("Enter request: ");  
for(j=0;j<m;j++) scanf("%d",&req[j]);
```

```
/* Request <= Need and Available */  
for(j=0;j<m;j++)  
    if(req[j]>need[p][j] || req[j]>avail[j]) {  
        printf("Request cannot be granted immediately.\n");
```

```

    return 0;
}

/* Temporary allocation */
for(j=0;j<m;j++) {
    avail[j]-=req[j];
    a[p][j]+=req[j];
    need[p][j]-=req[j];
    work[j]=avail[j];
}

for(i=0;i<n;i++) finish[i]=0;

/* Safety check */
for(int k=0;k<n;k++)
    for(i=0;i<n;i++)
        if(!finish[i]) {
            int ok=1;
            for(j=0;j<m;j++)
                if(need[i][j]>work[j]) ok=0;

            if(ok) {
                for(j=0;j<m;j++)
                    work[j]+=a[i][j];
                finish[i]=1;
            }
        }

for(i=0;i<n;i++)
    if(!finish[i]) {
        printf("Request cannot be granted.\n");
        return 0;
    }

printf("Request can be granted immediately.\n");
return 0;

```

}

Q.3. Oral/Viva

Savitribai Phule Pune University
T.Y.B.Sc.(C.S.) (NEP – 2020) SEM – V Practical Examination
Lab Course CS-305-MJ-P
Operating System-I Duration:

Q 1) Write a program to simulate Sequential file allocation method. Assume disk with n number of blocks. Give value of n as input. Randomly mark some block as allocated and accordingly maintain the list of free blocks Write menu driver program with menu options as mentioned below and implement each option.

- ☐ Show Bit Vector
- ☐ Create New File
- ☐ Show Directory
- ☐ Exit

Answer:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int n,i,ch,s,l,bit[50]={0};
    char file[20];

    printf("Enter blocks: ");
    scanf("%d",&n);

    /* Randomly allocate blocks */
    for(i=0;i<n;i++)
        bit[i]=rand()%2;

    while(1)
    {
```

```

printf("\n1.Bit Vector 2.New File 3.Directory 4.Exit");
printf("\nChoice: ");
scanf(" %d",&ch);

if(ch==1)
{
    for(i=0;i<n;i++) printf(" %d ",bit[i]);
}

else if(ch==2)
{
    printf("\nFile name, start, length: ");
    scanf(" %s%d%d",file,&s,&l);

    for(i=s;i<s+l;i++)
        if(i>=n || bit[i]) break;

    if(i==s+l)
    {
        for(i=s;i<s+l;i++) bit[i]=1;
        printf("File created.");
    }
    else printf("Blocks not free.");
}

else if(ch==3)
    printf("\nFile: %s Start: %d Length: %d",file,s,l);

else if(ch==4)
    break;

else
    printf("Invalid choice");
}
return 0;
}

```

Q 2) Write a simulation program for disk scheduling using C-SCAN algorithm. Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of requests in the order in which it is served. also display the total number of head movements (Assume disk size=200) 15, 45, 75, 105, 135, 165, 195, 80

Starting Head Position: 100

Direction: Right

Answer:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main()
```

```
{
```

```
    int a[20], n, head, i, j, temp, total = 0;
```

```
    printf("Enter number of requests: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter requests: ");
```

```
    for(i=0;i<n;i++) scanf("%d",&a[i]);
```

```
    printf("Enter head position: ");
```

```
    scanf("%d",&head);
```

```
    /* Sort requests */
```

```
    for(i=0;i<n-1;i++)
```

```
        for(j=i+1;j<n;j++)
```

```
            if(a[i]>a[j])
```

```
            {
```

```
                temp=a[i]; a[i]=a[j]; a[j]=temp;
```

```
            }
```

```

printf("\nService order: ");

/* Requests to the right */
for(i=0;i<n;i++)
    if(a[i]>=head)
    {
        printf("%d ",a[i]);
        total += abs(head-a[i]);
        head=a[i];
    }

/* Go to end of disk */
total += 199-head;
head=199;

/* C-SCAN jump to beginning */
total += 199;
head=0;

/* Requests from beginning */
for(i=0;i<n;i++)
    if(a[i]<head || a[i]<100)
    {
        printf("%d ",a[i]);
        total += abs(head-a[i]);
        head=a[i];
    }

printf("\nTotal head movement = %d\n",total);

return 0;
}

```

Q.3. Oral/Viva

Savitribai Phule Pune University
T.Y.B.Sc.(C.S.) (NEP – 2020) SEM – V Practical Examination
Lab Course CS-305-MJ-P
Operating System-I Duration:

Q 1) Write a program to simulate Index file allocation method. Assume disk with 'n' number of blocks. Give value of 'n' as input. Randomly mark some block as allocated and accordingly maintain the list of free blocks Write menu driven

program with menu options as mentioned above and implement each option

- ☐ Show Bit Vector
- ☐ Create New File
- ☐ Show Directory
- ☐ Exit

Answer:

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main()
{
    int n,b[50],i,ch,index,k,d[20];
    char name[20];

    printf("Enter blocks: ");
    scanf("%d",&n);

    for(i=0;i<n;i++) b[i]=rand()%2;

    while(1)
    {
        printf("\n1.Bit Vector 2.New File 3.Directory 4.Exit\n");
        scanf("%d",&ch);
```

```

if(ch==1)
{
    for(i=0;i<n;i++) printf(" %d ",b[i]);
}

else if(ch==2)
{
    printf("Name Index No.of blocks: ");
    scanf(" %s%d%d",name,&index,&k);

    if(b[index])
    {
        printf("Index block busy!");
        continue;
    }

    printf("Enter blocks: ");
    for(i=0;i<k;i++) scanf(" %d",&d[i]);

    for(i=0;i<k;i++)
        if(d[i]>=n || b[d[i]]) break;

    if(i==k)
    {
        b[index]=1;
        for(i=0;i<k;i++) b[d[i]]=1;
        printf("File created.");
    }
    else printf("Blocks not free.");
}

else if(ch==3)
{
    printf("File: %s\nIndex: %d\nBlocks: ",name,index);
    for(i=0;i<k;i++) printf(" %d ",d[i]);

```

```

    }

    else if(ch==4) exit(0);
}
}

```

Q 2) Write a program that demonstrates the use of nice () system call. After a child process is started using fork(), assign higher priority to the child using nice() system call.

Answer:

```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pid;

    pid = fork();

    if(pid == 0)
    {
        printf("Child process\n");
        printf("Old priority: %d\n", nice(0));

        /* Higher priority: decrease nice value */
        if(nice(-5) == -1)
            perror("nice");

        printf("New priority: %d\n", nice(0));
    }
    else
    {

```

```
    wait(NULL);  
    printf("Parent process\n");  
}  
  
return 0;  
}
```

Q.3. Oral/Viva

T.Y.B.Sc.(C.S.) (NEP – 2020) SEM – V Practical Examination
Lab Course CS-305-MJ-P
Operating System-I Duration:

Q 1) Write the simulation program for demand paging and show the page scheduling and total number of page faults according the optimal page replacement algorithm. Assume the memory of 'n' frames.

Reference String: 8, 5, 7, 8, 5, 7, 2, 3, 7, 3, 5, 9, 4, 6, 2

Answer:

```
#include <stdio.h>

int main()
{
    int p[] = {8,5,7,8,5,7,2,3,7,3,5,9,4,6,2};
    int n, f[10], i, j, k, pos, far, hit, faults=0;

    printf("Enter number of frames: ");
    scanf("%d",&n);

    for(i=0;i<n;i++) f[i]=-1;

    printf("\nPage\tFrames\t\tStatus\n");

    for(i=0;i<15;i++)
    {
        hit=0;

        for(j=0;j<n;j++)
            if(f[j]==p[i]) hit=1;

        if(!hit)
        {
            faults++;
        }
    }
}
```

```

/* Empty frame */
for(j=0;j<n;j++)
    if(f[j]==-1) break;

if(j<n)
    f[j]=p[i];
else
{
    far=-1;

    for(j=0;j<n;j++)
    {
        for(k=i+1;k<15;k++)
            if(f[j]==p[k]) break;

        if(k>far)
        {
            far=k;
            pos=j;
        }
    }
    f[pos]=p[i];
}

printf("%d\t",p[i]);
for(j=0;j<n;j++)
    printf("%d ",f[j]);
printf(hit ? "\tHit\n" : "\tFault\n");
}

printf("\nTotal Page Faults = %d\n",faults);

return 0;
}

```

Q 2) Write the program to simulate Non-preemptive Shortest Job First (SJF) scheduling. the arrival time and first CPU burst for different n number of processes should be input to the algorithm. The next CPU burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time.

Answer:

```
#include <stdio.h>
```

```
int main() {  
    int n, i, j, t=0, p[10], at[10], bt[10], wt[10], tat[10], done[10]={0};  
    float aw=0, atat=0;
```

```
    printf("Enter n: ");  
    scanf("%d",&n);
```

```
    for(i=0;i<n;i++) {  
        p[i]=i+1;  
        printf("AT BT of P%d: ",p[i]);  
        scanf("%d%d",&at[i],&bt[i]);  
    }
```

```
    printf("\nGantt Chart: ");  
    for(i=0;i<n;i++) {  
        int k=-1;  
        for(j=0;j<n;j++)  
            if(!done[j] && at[j]<=t && (k==-1 || bt[j]<bt[k]))  
                k=j;
```

```
        if(k==-1) { t++; i--; continue; }
```

```
        printf("| P%d ",p[k]);  
        wt[k]=t-at[k];  
        t+=bt[k];
```

```

    tat[k]=t-at[k];
    done[k]=1;
}

printf("\n\nP\tWT\tTAT\n");
for(i=0;i<n;i++) {
    printf("P%d\t%d\t%d\n",p[i],wt[i],tat[i]);
    aw+=wt[i];
    atat+=tat[i];
}

printf("\nAvg WT = %.2f",aw/n);
printf("\nAvg TAT = %.2f",atat/n);
return 0;
}

```

Q.3. Oral/Viva

Savitribai Phule Pune University
T.Y.B.Sc.(C.S.) (NEP – 2020) SEM – V Practical Examination
Lab Course CS-305-MJ-P
Operating System-I Duration:

Q 1) Write the program to simulate Non-preemptive Shortest Job First (SJF) scheduling. The arrival time and first CPU burst for different n number of processes should be input to the algorithm. The next CPU-burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time.

Answer:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
```

```
#define MAX 100
```

```
struct Process
{
    int pid;
    int arrival;
    int burst1;
    int burst2;

    int completion;
    int turnaround;
    int waiting;

    int completed;
};
```

```

int main()
{
    struct Process p[MAX];
    int n;
    int currentTime = 0;
    int completed = 0;
    int i;

    float avgWaiting = 0;
    float avgTurnaround = 0;

    srand(time(NULL));

    printf("Enter number of processes: ");
    scanf("%d", &n);

    if (n <= 0 || n > MAX)
    {
        printf("Invalid number of processes.\n");
        return 1;
    }

    /* Input arrival time and first CPU burst */
    for (i = 0; i < n; i++)
    {
        p[i].pid = i + 1;

        printf("\nEnter arrival time of P%d: ", i + 1);
        scanf("%d", &p[i].arrival);

        printf("Enter first CPU burst of P%d: ", i + 1);
        scanf("%d", &p[i].burst1);

        /* Generate second CPU burst randomly: 1 to 10 */
        p[i].burst2 = (rand() % 10) + 1;
    }
}

```

```

    p[i].completed = 0;
}

/* Display generated second CPU bursts */
printf("\nRandomly generated second CPU bursts:\n");

for (i = 0; i < n; i++)
{
    printf("P%d = %d\n", p[i].pid, p[i].burst2);
}

/*
 * Non-preemptive SJF scheduling.
 *
 * A process is selected based on the shortest
 * first CPU burst among the processes that have arrived.
 */
while (completed < n)
{
    int shortest = -1;
    int shortestBurst = 99999;

    /* Find shortest available process */
    for (i = 0; i < n; i++)
    {
        if (p[i].completed == 0 &&
            p[i].arrival <= currentTime)
        {
            if (p[i].burst1 < shortestBurst)
            {
                shortestBurst = p[i].burst1;
                shortest = i;
            }
        }
    }
}

```

```

/* If no process has arrived, move time forward */
if (shortest == -1)
{
    int nextArrival = 99999;

    for (i = 0; i < n; i++)
    {
        if (p[i].completed == 0 &&
            p[i].arrival < nextArrival)
        {
            nextArrival = p[i].arrival;
        }
    }

    currentTime = nextArrival;
    continue;
}

/*
 * Execute first CPU burst.
 */
currentTime += p[shortest].burst1;

/*
 * Execute randomly generated second CPU burst.
 */
currentTime += p[shortest].burst2;

/* Completion time */
p[shortest].completion = currentTime;

/* Turnaround Time */
p[shortest].turnaround =
    p[shortest].completion - p[shortest].arrival;

/* Waiting Time */

```

```

    p[shortest].waiting =
        p[shortest].turnaround -
        (p[shortest].burst1 + p[shortest].burst2);

    p[shortest].completed = 1;

    completed++;
}

/* Calculate averages */
for (i = 0; i < n; i++)
{
    avgWaiting += p[i].waiting;
    avgTurnaround += p[i].turnaround;
}

avgWaiting /= n;
avgTurnaround /= n;

/* Display Gantt Chart */
printf("\n\n===== GANTT CHART =====\n\n");

/*
 * Sort processes according to completion time
 * only for displaying the execution order.
 */
{
    int order[MAX];
    int temp;

    for (i = 0; i < n; i++)
    {
        order[i] = i;
    }

    for (i = 0; i < n - 1; i++)

```

```

{
    int j;

    for (j = i + 1; j < n; j++)
    {
        if (p[order[i]].completion >
            p[order[j]].completion)
        {
            temp = order[i];
            order[i] = order[j];
            order[j] = temp;
        }
    }
}

for (i = 0; i < n; i++)
{
    printf("| P%d  ", p[order[i]].pid);
}

printf("\n");
}

/* Display process details */
printf("\n\n===== PROCESS DETAILS
=====\\n");

printf("\n%-5s  %-8s  %-8s  %-8s  %-12s  %-12s  %-10s\\n",
    "PID",
    "Arrival",
    "Burst1",
    "Burst2",
    "Completion",
    "Turnaround",
    "Waiting");

```

```

printf("-----\n");

for (i = 0; i < n; i++)
{
    printf("%-5d %-8d %-8d %-8d %-12d %-12d %-10d\n",
        p[i].pid,
        p[i].arrival,
        p[i].burst1,
        p[i].burst2,
        p[i].completion,
        p[i].turnaround,
        p[i].waiting);
}

printf("\n=====
\n");
    printf("Average Waiting Time   : %.2f\n", avgWaiting);
    printf("Average Turnaround Time : %.2f\n", avgTurnaround);

printf("=====
");

    return 0;
}

```

Q 2) Write a simulation program for disk scheduling using LOOK algorithm.

Accept total number of disk blocks, disk request string, and current head position from the user. isplay the list of requests in the order in which it is served.

Also display the total number of head moments

15, 30, 55, 90, 125, 140, 170

Starting Head Position: 100

Direction: Left

Answer:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
void sort(int arr[], int n)
```

```
{
```

```
    int i, j, temp;
```

```
    for (i = 0; i < n - 1; i++)
```

```
    {
```

```
        for (j = i + 1; j < n; j++)
```

```
        {
```

```
            if (arr[i] > arr[j])
```

```
            {
```

```
                temp = arr[i];
```

```
                arr[i] = arr[j];
```

```
                arr[j] = temp;
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

```
int main()
```

```
{
```

```
    int totalBlocks;
```

```
    int n;
```

```
    int request[100];
```

```
    int head;
```

```
    int i;
```

```
    int totalMovement = 0;
```

```
    char direction;
```

```
    printf("Enter total number of disk blocks: ");
```

```
    scanf("%d", &totalBlocks);
```

```

printf("Enter number of disk requests: ");
scanf("%d", &n);

printf("Enter disk request string:\n");

for (i = 0; i < n; i++)
{
    scanf("%d", &request[i]);

    if (request[i] < 0 || request[i] >= totalBlocks)
    {
        printf("Invalid request: %d\n", request[i]);
        return 1;
    }
}

printf("Enter current head position: ");
scanf("%d", &head);

printf("Enter direction (L/R): ");
scanf(" %c", &direction);

if (head < 0 || head >= totalBlocks)
{
    printf("Invalid head position.\n");
    return 1;
}

/* Sort requests */
sort(request, n);

printf("\nRequests in sorted order: ");
for (i = 0; i < n; i++)
{
    printf("%d ", request[i]);
}

```

```
printf("\n");
```

```
printf("\nOrder in which requests are served:\n");  
printf(" %d", head);
```

```
/*
```

```
 * LOOK scheduling:
```

```
 * Move in the selected direction until the last
```

```
 * request in that direction is served, then reverse.
```

```
*/
```

```
if (direction == 'L' || direction == 'I')
```

```
{
```

```
    /* Serve requests smaller than head, from nearest to farthest */
```

```
    for (i = n - 1; i >= 0; i--)
```

```
    {
```

```
        if (request[i] < head)
```

```
        {
```

```
            totalMovement += abs(head - request[i]);
```

```
            head = request[i];
```

```
            printf(" -> %d", head);
```

```
        }
```

```
    }
```

```
    /* Reverse direction and serve remaining requests */
```

```
    for (i = 0; i < n; i++)
```

```
    {
```

```
        if (request[i] > head)
```

```
        {
```

```
            totalMovement += abs(head - request[i]);
```

```
            head = request[i];
```

```
            printf(" -> %d", head);
```

```
        }
```

```

    }
}
else if (direction == 'R' || direction == 'r')
{
    /* Serve requests greater than head, nearest first */
    for (i = 0; i < n; i++)
    {
        if (request[i] > head)
        {
            totalMovement += abs(head - request[i]);
            head = request[i];

            printf(" -> %d", head);
        }
    }

    /* Reverse direction and serve remaining requests */
    for (i = n - 1; i >= 0; i--)
    {
        if (request[i] < head)
        {
            totalMovement += abs(head - request[i]);
            head = request[i];

            printf(" -> %d", head);
        }
    }
}
else
{
    printf("\nInvalid direction.\n");
    return 1;
}

printf("\n\nTotal head movement = %d cylinders\n", totalMovement);

```

```
    return 0;  
}
```

Q.3. Oral/Viva

Savitribai Phule Pune University
T.Y.B.Sc.(C.S.) (NEP – 2020) SEM – V Practical Examination
Lab Course CS-305-MJ-P
Operating System-I Duration:

Q 1) Write a C program to illustrate the concept of orphan process. Parent process creates a child and terminates before child has finished its task. So child process becomes orphan process. (Use fork(), sleep(), getpid(), getppid())

Answer:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <stdlib.h>

int main()
{
    pid_t pid;

    /* Create child process */
    pid = fork();

    if (pid < 0)
    {
        printf("Fork failed.\n");
        return 1;
    }

    /* Child process */
    if (pid == 0)
    {
        printf("Child process started.\n");
        printf("Child PID : %d\n", getpid());
```

```

printf("Parent PID : %d\n", getppid());

/* Child continues working */
printf("\nChild is performing its task...\n");
sleep(5);

/*
 * Parent has already terminated.
 * Therefore, child's parent PID changes.
 */
printf("\nChild process after parent termination:\n");
printf("Child PID : %d\n", getpid());
printf("Parent PID : %d\n", getppid());

printf("Child process completed.\n");
}

/* Parent process */
else
{
    printf("Parent process started.\n");
    printf("Parent PID : %d\n", getpid());
    printf("Child PID : %d\n", pid);

    /*
     * Parent terminates before child finishes.
     */
    printf("\nParent process is terminating...\n");

    sleep(2);
    exit(0);
}

return 0;
}

```

Q 2) Write a C program that behaves like a shell which displays the command prompt '\$'. It accepts the command, tokenize the command line and execute it by creating the child process. Also implement the additional command 'search' as

a. \$ search f filename pattern : It will search the first occurrence of pattern in the given file

b. \$ search a filename pattern : It will search all the occurrence of pattern in the given file

c. \$ search c filename pattern : It will count the number of occurrence of pattern in the given file

Answer:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
```

```
#define MAX 100
#define SIZE 1024
```

```
/* Search first occurrence */
void search_first(char *filename, char *pattern)
{
    FILE *fp;
    char line[SIZE];
    int lineNo = 0;

    fp = fopen(filename, "r");

    if (fp == NULL)
    {
        perror("File");
        return;
    }
}
```

```

while (fgets(line, sizeof(line), fp) != NULL)
{
    lineNo++;

    if (strstr(line, pattern) != NULL)
    {
        printf("Pattern found at line %d\n", lineNo);
        printf("%s", line);
        fclose(fp);
        return;
    }
}

printf("Pattern not found.\n");

fclose(fp);
}

/* Search all occurrences */
void search_all(char *filename, char *pattern)
{
    FILE *fp;
    char line[SIZE];
    char *pos;
    int lineNo = 0;
    int found = 0;

    fp = fopen(filename, "r");

    if (fp == NULL)
    {
        perror("File");
        return;
    }
}

```

```

while (fgets(line, sizeof(line), fp) != NULL)
{
    lineNo++;
    pos = line;

    while ((pos = strstr(pos, pattern)) != NULL)
    {
        printf("Pattern found at line %d, position %ld\n",
            lineNo, (long)(pos - line + 1));

        found = 1;
        pos++;
    }
}

if (!found)
    printf("Pattern not found.\n");

fclose(fp);
}

/* Count occurrences */
void search_count(char *filename, char *pattern)
{
    FILE *fp;
    char line[SIZE];
    char *pos;
    int count = 0;

    fp = fopen(filename, "r");

    if (fp == NULL)
    {
        perror("File");
        return;
    }
}

```

```
while (fgets(line, sizeof(line), fp) != NULL)
{
    pos = line;

    while ((pos = strstr(pos, pattern)) != NULL)
    {
        count++;
        pos++;
    }
}

printf("Number of occurrences = %d\n", count);

fclose(fp);
}
```

```
int main()
{
    char command[SIZE];
    char *args[MAX];
    int argc;
    pid_t pid;

    while (1)
    {
        printf("$ ");
        fflush(stdout);

        /* Read command */
        if (fgets(command, sizeof(command), stdin) == NULL)
            break;

        /* Remove newline */
        command[strcspn(command, "\n")] = '\0';
```

```

/* Ignore empty command */
if (strlen(command) == 0)
    continue;

/* Tokenize command */
argc = 0;

args[argc] = strtok(command, " ");

while (args[argc] != NULL && argc < MAX - 1)
{
    argc++;
    args[argc] = strtok(NULL, " ");
}

args[argc] = NULL;

/* Exit command */
if (strcmp(args[0], "exit") == 0)
{
    printf("Shell terminated.\n");
    break;
}

/*
 * Additional search command
 *
 * search f filename pattern
 * search a filename pattern
 * search c filename pattern
 */
if (strcmp(args[0], "search") == 0)
{
    if (argc != 4)
    {
        printf("Usage:\n");
    }
}

```

```

        printf("search f filename pattern\n");
        printf("search a filename pattern\n");
        printf("search c filename pattern\n");
        continue;
    }

    if (strcmp(args[1], "f") == 0)
    {
        search_first(args[2], args[3]);
    }
    else if (strcmp(args[1], "a") == 0)
    {
        search_all(args[2], args[3]);
    }
    else if (strcmp(args[1], "c") == 0)
    {
        search_count(args[2], args[3]);
    }
    else
    {
        printf("Invalid search option.\n");
    }

    continue;
}

/*
 * Execute normal Linux commands
 * using a child process.
 */
pid = fork();

if (pid < 0)
{
    perror("fork");
    continue;
}

```

```
}

if (pid == 0)
{
    /* Child process */
    execvp(args[0], args);

    /* execvp returns only if an error occurs */
    perror("Command not found");
    exit(1);
}
else
{
    /* Parent waits for child */
    wait(NULL);
}
}

return 0;
}
```

Q.3. Oral/Viva

Savitribai Phule Pune University
T.Y.B.Sc.(C.S.) (NEP – 2020) SEM – V Practical Examination
Lab Course CS-305-MJ-P
Operating System-I Duration:

Q 1) Write a simulation program for disk scheduling using SSTF algorithm. Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of requests in the order in which it is served. Also display the total number of head moments.

30, 10, 60, 95, 120, 150, 175

Answer:

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main()
{
    int n;
    int request[100];
    int visited[100] = {0};
    int head;
    int totalMovement = 0;
    int i, j;

    printf("Enter total number of disk blocks: ");
    scanf("%d", &n);

    if (n <= 0 || n > 1000)
    {
        printf("Invalid number of disk blocks.\n");
        return 1;
    }
}
```

```
printf("Enter number of disk requests: ");
int requestCount;
scanf("%d", &requestCount);

if (requestCount <= 0 || requestCount > 100)
{
    printf("Invalid number of requests.\n");
    return 1;
}

printf("Enter disk request string:\n");

for (i = 0; i < requestCount; i++)
{
    scanf("%d", &request[i]);

    if (request[i] < 0 || request[i] >= n)
    {
        printf("Invalid request: %d\n", request[i]);
        return 1;
    }
}

printf("Enter starting head position: ");
scanf("%d", &head);

if (head < 0 || head >= n)
{
    printf("Invalid head position.\n");
    return 1;
}

printf("\nOrder in which requests are served:\n");
printf("%d", head);

/*
```

```

* SSTF:
* Select the unserved request having the
* shortest distance from the current head.
*/
for (i = 0; i < requestCount; i++)
{
    int shortest = -1;
    int minDistance = 99999;

    for (j = 0; j < requestCount; j++)
    {
        if (visited[j] == 0)
        {
            int distance = abs(head - request[j]);

            if (distance < minDistance)
            {
                minDistance = distance;
                shortest = j;
            }
        }
    }

    /* Serve selected request */
    visited[shortest] = 1;

    totalMovement += abs(head - request[shortest]);

    head = request[shortest];

    printf(" -> %d", head);
}

printf("\n\nTotal head movement = %d blocks\n",
    totalMovement);

```

```
    return 0;
}
```

Q 2) Write the simulation program to implement demand paging and show the page scheduling and total number of page faults according to the LRU (using counter method) page replacement algorithm. Assume the memory of n frames.

Reference String : 3,5,7,2,5,1,2,3,1,3,5,3,1,6,2

Answer:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int frames[20];
```

```
    int counter[20];
```

```
    int reference[] = {3, 5, 7, 2, 5, 1, 2, 3, 1, 3, 5, 3, 1, 6, 2};
```

```
    int n;
```

```
    int refSize = 15;
```

```
    int pageFaults = 0;
```

```
    int time = 0;
```

```
    int i, j;
```

```
    int found;
```

```
    int position;
```

```
    int lru;
```

```
    printf("Enter number of frames: ");
```

```
    scanf("%d", &n);
```

```
    if (n <= 0 || n > 20)
```

```
    {
```

```
        printf("Invalid number of frames.\n");
```

```
        return 1;
```

```
}
```

```
/* Initialize frames and counters */
```

```
for (i = 0; i < n; i++)
```

```
{
```

```
    frames[i] = -1;
```

```
    counter[i] = 0;
```

```
}
```

```
printf("\nReference String:\n");
```

```
printf("3 5 7 2 5 1 2 3 1 3 5 3 1 6 2\n");
```

```
printf("\n===== LRU PAGE REPLACEMENT  
===== \n\n");
```

```
printf("Page\t");
```

```
for (i = 0; i < n; i++)
```

```
{
```

```
    printf("F%d\t", i + 1);
```

```
}
```

```
printf("Status\n");
```

```
/* Process reference string */
```

```
for (i = 0; i < refSize; i++)
```

```
{
```

```
    found = 0;
```

```
    position = -1;
```

```
    time++;
```

```
/* Check if page is already present */
```

```
for (j = 0; j < n; j++)
```

```
{
```

```
    if (frames[j] == reference[i])
```

```

    {
        found = 1;
        position = j;
        break;
    }
}

if (found)
{
    /*
     * Page hit:
     * Update counter to current time.
     */
    counter[position] = time;

    printf("%d\t", reference[i]);

    for (j = 0; j < n; j++)
    {
        if (frames[j] == -1)
            printf("-\t");
        else
            printf("%d\t", frames[j]);
    }

    printf("Hit\n");
}
else
{
    /* Page fault */
    pageFaults++;

    /*
     * First look for an empty frame.
     */
    position = -1;

```

```

for (j = 0; j < n; j++)
{
    if (frames[j] == -1)
    {
        position = j;
        break;
    }
}

/*
 * If no empty frame exists,
 * find the least recently used page.
 */
if (position == -1)
{
    lru = 0;

    for (j = 1; j < n; j++)
    {
        if (counter[j] < counter[lru])
        {
            lru = j;
        }
    }

    position = lru;
}

/* Replace page */
frames[position] = reference[i];

/* Update counter */
counter[position] = time;

printf("%d\t", reference[i]);

```

```

    for (j = 0; j < n; j++)
    {
        if (frames[j] == -1)
            printf("-\t");
        else
            printf("%d\t", frames[j]);
    }

    printf("Page Fault\n");
}

printf("\nTotal Number of Page Faults = %d\n", pageFaults);

return 0;
}

```

Q.3. Oral/Viva

